A Class-Based Ride Sharing System in C++ and Smalltalk

Xhon Pelushi

University of the Cumberlands

MSCS-632-M30: Advanced Programming Languages

Dr. Jay Thom

June 19, 2026

Abstract

This report describes a class-based ride sharing system implemented independently in C++ and Smalltalk (Pharo) to compare how each language expresses the three core object-oriented principles of encapsulation, inheritance, and polymorphism. Both implementations define a Ride base class with StandardRide and PremiumRide subclasses that each price a trip differently, plus Driver and Rider classes that manage private collections of rides. Running both programs against the same sample data produced matching fares and matching polymorphic behavior, while the source code itself shows that C++ enforces these principles through explicit keywords (private, virtual, override) checked at compile time, whereas Smalltalk treats encapsulation and dynamic dispatch as unconditional defaults of its object model.

Overview

The system models a simplified ride sharing platform: a Ride base class stores a ride ID, pickup and dropoff locations, and a distance, and exposes a fare() method and a rideDetails() method. StandardRide and PremiumRide both inherit from Ride and override fare() with their own per-mile pricing, and a small driver program stores several rides of mixed types in a single list to demonstrate calling fare() and rideDetails() polymorphically. A Driver class and a Rider class each keep a private list of rides (assignedRides and requestedRides, respectively) that can only be modified or summarized through their own methods. C++ and Smalltalk were chosen because they sit at opposite ends of the object-oriented design spectrum: C++ is statically typed and compiles object behavior into vtables, while Smalltalk is dynamically typed and resolves every method call by sending a message at runtime (Sebesta, 2019).

Encapsulation

In C++, encapsulation is an opt-in access-control feature. Ride's rideID, pickupLocation, dropoffLocation, and distance fields, and Driver's and Rider's assignedRides/requestedRides collections, are all declared private; the compiler rejects any code outside the class that tries to

touch them directly, and the only sanctioned access is through public getter methods or through addRide()/requestRide() and getDriverInfo()/viewRides() (Stroustrup, 2013). A programmer could still choose to make those fields public, so the protection is a deliberate design choice rather than a property of the language itself.

Smalltalk makes the same guarantee unconditionally. Every instance variable - rideID, pickupLocation, assignedRides, and so on - is private to its defining class by construction; there is no syntax in the language that lets one object reach into another object's instance variables, even accidentally (Black et al., 2009). The only way to interact with a Ride, Driver, or Rider is to send it a message, which means encapsulation in Smalltalk is not a keyword a programmer applies but a consequence of the language having no other way to access state.

Inheritance

C++ expresses inheritance through the class StandardRide : public Ride syntax, which is resolved at compile time: the compiler lays out StandardRide's memory so that it contains every field Ride declared, plus a vtable pointer used to resolve overridden virtual methods. Smalltalk expresses the same relationship as a message sent to a class - Ride subclass: #StandardRide instanceVariableNames: ... - which creates the subclass at runtime rather than at compile time. Both approaches produce single inheritance with the same practical result (StandardRide and PremiumRide both reuse Ride's constructor logic and accessors), but Smalltalk's version is itself just another message send, consistent with the language's premise that classes are themselves objects that understand messages (Goldberg & Robson, 1983, as cited in Black et al., 2009).

Polymorphism

Polymorphism is where the two languages diverge the most. In C++, fare() and rideDetails() had to be declared virtual in Ride and override in StandardRide and PremiumRide; without those keywords, calling ride->fare() through a base-class pointer would statically bind to Ride's own implementation no matter what the underlying object actually was. With virtual in

place, the program's main loop can hold a std::vector<std::unique_ptr<Ride>> containing a mix of StandardRide and PremiumRide objects and call fare() on each one, and the vtable look-up dispatches to the correct override every time.

Smalltalk has no separate concept of a virtual method, because every method call is already a dynamically dispatched message: sending #fare to a StandardRide and to a PremiumRide simply looks up #fare in each object's actual class at the moment the message arrives, with no compile-time binding step to opt out of. The demo script's rides do: [:r | Transcript show: r rideDetails] loop works for exactly this reason - it is not 'using polymorphism,' it is simply how Smalltalk always calls methods. Running both programs against identical sample data confirmed the two languages produce the same fares and the same driver/rider summaries, which shows that C++'s explicit virtual dispatch and Smalltalk's universal message dispatch are two different mechanisms reaching the same observable behavior.

Conclusion

Building the same Ride Sharing System twice showed that encapsulation, inheritance, and polymorphism are language-agnostic design principles, but the two languages enforce them very differently. C++ requires the programmer to opt into each principle explicitly - private for encapsulation, virtual/override for polymorphism - and checks those choices at compile time, while Smalltalk treats all three as unconditional defaults of its message-passing object model. Neither approach is strictly better: C++'s explicitness catches mistakes earlier and avoids any dispatch overhead when polymorphism is not needed, while Smalltalk's uniformity means a programmer never has to remember to ask for the behavior they want.

Source Code

The complete source code for both implementations is available in the public GitHub repository: https://github.com/xhon-pelushi/MSCS-632-Advanced-Programming-Languages/tree/main/Assignment-5

References

Black, A., Ducasse, S., Nierstrasz, O., & Pollet, D. (2009). Pharo by example. Square Bracket Associates.

Sebesta, R. W. (2019). Concepts of programming languages (12th ed.). Pearson.

Stroustrup, B. (2013). The C++ programming language (4th ed.). Addison-Wesley.

Appendix: Screenshots

C++ Source - Ride base class and subclasses

```
// ---- Ride.h ----
#ifndef RIDE_H
#define RIDE_H

#include <string>

class Ride {
public:
    Ride(int rideID, std::string pickupLocation, std::string dropoffLocation, double distance);
    virtual ~Ride() = default;

    int getRideID() const;
    const std::string& getPickupLocation() const;
    const std::string& getDropoffLocation() const;
    double getDistance() const;

    // Polymorphic fare calculation; each ride type prices distance differently.
    virtual double fare() const;

    // Polymorphic display of ride information, built on top of fare().
    virtual std::string rideDetails() const;

protected:
    virtual std::string rideType() const;

private:
    int rideID;
    std::string pickupLocation;
    std::string dropoffLocation;
    double distance;
};

#endif

// ---- StandardRide.cpp ----
#include "StandardRide.h"

StandardRide::StandardRide(int rideID, std::string pickupLocation, std::string dropoffLocation, double distance)
    : Ride(rideID, std::move(pickupLocation), std::move(dropoffLocation), distance) {}

double StandardRide::fare() const {
    const double baseFare = 2.50;
    const double perMile = 1.25;
    return baseFare + perMile * getDistance();
}

std::string StandardRide::rideType() const {
    return "StandardRide";
}

// ---- PremiumRide.cpp ----
```

Figure 1. C++ source - Ride base class with virtual fare()/rideDetails(), and the StandardRide/PremiumRide overrides.

C++ (g++ 13.3) - Ride Sharing System Output

```
$ ./ride_sharing
=== Ride Sharing System (C++) ===

-- Polymorphic ride list (fare() and rideDetails()) --
[StandardRide #101] Maple St -> Oak Ave (4.2 mi), fare: $7.75
[PremiumRide #102] Airport -> Downtown (12.8 mi), fare: $40.00
[StandardRide #103] 5th Ave -> Central Park (2.5 mi), fare: $5.62
[PremiumRide #104] Hotel Plaza -> Convention Center (6.0 mi), fare: $23.00

-- Driver summary --
Driver #1 Jordan Lee (rating 4.9) - 3 ride(s) completed:
    [StandardRide #101] Maple St -> Oak Ave (4.2 mi), fare: $7.75
    [PremiumRide #102] Airport -> Downtown (12.8 mi), fare: $40.00
    [PremiumRide #104] Hotel Plaza -> Convention Center (6.0 mi), fare: $23.00
    Total earnings: $70.75

-- Rider summary --
Rider #1 Avery Chen - 2 ride(s) requested:
    [StandardRide #101] Maple St -> Oak Ave (4.2 mi), fare: $7.75
    [StandardRide #103] 5th Ave -> Central Park (2.5 mi), fare: $5.62
```

Figure 2. C++ program output: polymorphic ride list, driver summary, and rider summary.

Pharo Smalltalk Source - Ride base class and subclasses

```
"---- RideSharingSystem.st ----"
"Ride Sharing System - Pharo Smalltalk
Run headlessly with:
    pharo --headless Pharo.image RideSharingSystem.st
"

"==================== Ride (base class) ===================="
Object subclass: #Ride
    instanceVariableNames: 'rideID pickupLocation dropoffLocation distance'
    classVariableNames: ''
    package: 'RideSharing'.

Ride class compile: 'id: anID pickup: aPickup dropoff: aDropoff distance: aDistance
    | r |
    r := self new.
    r setId: anID pickup: aPickup dropoff: aDropoff distance: aDistance.
    ^ r'.

"Encapsulation: rideID/pickupLocation/dropoffLocation/distance are private
instance variables. Outside code can only read them through these accessor
methods, and can only set them through setId:pickup:dropoff:distance:."
Ride compile: 'setId: anID pickup: aPickup dropoff: aDropoff distance: aDistance
    rideID := anID.
    pickupLocation := aPickup.
    dropoffLocation := aDropoff.
    distance := aDistance'.

Ride compile: 'rideID
    ^ rideID'.

Ride compile: 'pickupLocation
    ^ pickupLocation'.

Ride compile: 'dropoffLocation
    ^ dropoffLocation'.

Ride compile: 'distance
    ^ distance'.

"Base fare formula; StandardRide and PremiumRide override this."
Ride compile: 'fare
    ^ (2.00 + (1.00 * distance)) roundTo: 0.01'.

"Overridden by each subclass so rideDetails stays polymorphic."
Ride compile: 'rideType
    ^ ''Ride'''.

Ride compile: 'rideDetails
    ^ ''['', self rideType, '' #'', rideID printString, ''] '',
        pickupLocation, '' -> '', dropoffLocation,
        '' ('', distance printString, '' mi), fare: $'', self fare printString'.

"==================== StandardRide (subclass) ===================="
```

Figure 3. Pharo Smalltalk source - Ride base class and the StandardRide/PremiumRide subclasses.

Pharo 11 Smalltalk - Ride Sharing System Output

```
$ pharo --headless Pharo.image RideSharingSystem.st
=== Ride Sharing System (Pharo Smalltalk) ===

-- Polymorphic ride list (fare and rideDetails) --
[StandardRide #101] Maple St -> Oak Ave (4.2 mi), fare: $7.75
[PremiumRide #102] Airport -> Downtown (12.8 mi), fare: $40.0
[StandardRide #103] 5th Ave -> Central Park (2.5 mi), fare: $5.63
[PremiumRide #104] Hotel Plaza -> Convention Center (6.0 mi), fare: $23.0

-- Driver summary --
Driver #1 Jordan Lee (rating 4.9) - 3 ride(s) completed:
    [StandardRide #101] Maple St -> Oak Ave (4.2 mi), fare: $7.75
    [PremiumRide #102] Airport -> Downtown (12.8 mi), fare: $40.0
    [PremiumRide #104] Hotel Plaza -> Convention Center (6.0 mi), fare: $23.0
    Total earnings: $70.75

-- Rider summary --
Rider #1 Avery Chen - 2 ride(s) requested:
    [StandardRide #101] Maple St -> Oak Ave (4.2 mi), fare: $7.75
    [StandardRide #103] 5th Ave -> Central Park (2.5 mi), fare: $5.63
```

Figure 4. Pharo Smalltalk program output, run headlessly, for the same sample data.